

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 4: Theoretical Questions

Foreign Trade University

Instructions

Part A contains 22 questions requiring written explanations or short proofs. Part B is a 10-question quiz (true/false and multiple choice). Attempt every question on your own before consulting Part C (Solutions) at the end of this document.

Part A: Theory Questions

- A1.** What is a *greedy algorithm*? Describe, in general terms, the structure of a greedy algorithm: what choice does it make at each step, and what does it never do afterward?
- A2.** Greedy algorithms are attractive because they are usually simple and efficient, but this efficiency comes with a risk. Explain what can go wrong if a greedy rule is applied without a correctness proof, and why “it worked on my test cases” is not sufficient justification for a greedy algorithm.
- A3.** Describe the “*greedy stays ahead*” proof technique in general terms: what quantity do we compare between the greedy solution and an arbitrary optimal solution, and what do we conclude by induction?
- A4.** Describe the *exchange argument* proof technique in general terms: starting from an arbitrary optimal solution O , what transformation is applied to O , and what must be shown about the effect of this transformation on O ’s objective value?
- A5.** State the Interval Scheduling Problem precisely: what is the input (in terms of start and finish times), what is a *feasible* (compatible) set of requests, and what is the objective?
- A6.** List three natural greedy heuristics for Interval Scheduling that *do not* always produce an optimal solution (e.g. based on start time, length, or number of conflicts), and for each, briefly describe the kind of instance that defeats it.
- A7.** State the Earliest-Finish-Time-First (EFT) algorithm for Interval Scheduling. Why is sorting by *finish* time (rather than start time) the key idea that makes this greedy rule work?
- A8.** Prove that the Earliest-Finish-Time-First algorithm produces an optimal solution to Interval Scheduling, using the “greedy stays ahead” technique. State precisely what is being compared at each step of the induction.
- A9.** State the Interval Partitioning (Interval Graph Coloring) Problem precisely: what is the input, what is a *resource assignment*, what makes an assignment *valid*, and what is the objective?

- A10.** Define the *depth* of a set of intervals. Prove that no valid resource assignment can use fewer than depth resources (i.e. depth is a lower bound on the optimum).
- A11.** State the Earliest-Start-Time-First algorithm for Interval Partitioning. Prove that it never needs more than depth resources, and conclude (using Q10) that it is optimal.
- A12.** State the Scheduling to Minimize Lateness Problem precisely: what is the input (in terms of processing times and deadlines), what is a *schedule*, how is the *lateness* of a job defined, and what is the objective?
- A13.** State the Earliest-Deadline-First (EDF) algorithm for Minimize Lateness. Give an example showing that Shortest-Job-First (sorting by processing time t_i instead of deadline d_i) can produce a strictly larger maximum lateness than EDF.
- A14.** Define an *inversion* in a schedule (with respect to deadlines) and define *idle time*. Prove that an optimal schedule for Minimize Lateness can always be chosen to have no idle time and no inversions.
- A15.** Using Q14, complete the proof that Earliest-Deadline-First is optimal for Minimize Lateness. Your argument should explain why EDF itself has no inversions and no idle time, and why any schedule with no inversions and no idle time has the same maximum lateness as any other such schedule (up to the specific job achieving the maximum).
- A16.** State the Optimal (Offline) Caching Problem precisely: what is the input (a sequence of page requests and a cache of size k), what is a *cache miss*, and what is the objective?
- A17.** State Belady’s Furthest-In-Future (FIF) algorithm. Explain, at a high level, why evicting the page whose next use is furthest in the future (or that is never used again) is the “safe” eviction choice – i.e. why an exchange argument can convert any optimal schedule into one that makes the same eviction at this step without increasing the number of misses.
- A18.** Define the Least-Recently-Used (LRU) eviction policy. Explain why LRU is a reasonable *online* heuristic (no knowledge of the future) even though it is not always optimal, and give a short example sequence where LRU and FIF make different eviction decisions.
- A19.** The Coin Change problem asks for the minimum number of coins (with unlimited supply of each denomination) summing to a target amount V . The greedy algorithm repeatedly takes the largest denomination $\leq$ the remaining amount. Define what it means for a coin system to be *canonical* (greedy-optimal for all amounts), and explain why the standard system $\{1, 5, 10, 25\}$ (US cents) is canonical while $\{1, 3, 4\}$ is not (give the smallest counter-example amount for $\{1, 3, 4\}$).
- A20.** State the Job Sequencing with Deadlines and Profits problem: each job i has a deadline d_i and profit p_i , all jobs take one unit of time, and a job earns its profit only if scheduled at or before its deadline on a single machine. Describe the greedy algorithm (sort by profit, then try to place each job as late as possible before its deadline) and explain at a high level why processing jobs in decreasing order of profit and placing each as late as possible is the right greedy rule.
- A21.** Compare the four “flavors” of greedy correctness seen this week: (a) Interval Scheduling (EFT, exchange/stays-ahead), (b) Interval Partitioning (depth lower bound + matching upper bound), (c) Minimize Lateness (exchange argument removing inversions), and (d) Optimal

Caching (exchange argument on eviction choice). What do all four proofs have in common, structurally?

- A22.** *Discussion (open-ended).* Suppose you are given a new optimization problem and you come up with a greedy rule that seems to work on every small example you try by hand. Describe a systematic process (drawing on the techniques in this chapter) you would follow to either (a) gain confidence that the rule is correct and find a proof, or (b) discover a counter-example that shows it is wrong. Be specific about what kinds of instances you would try to construct.

Part B: Quiz

- B1. (True/False)** A greedy algorithm, once it makes a choice, may later revisit and revise that choice if a better option becomes available.
- B2. (Multiple choice)** The Earliest-Finish-Time-First algorithm for Interval Scheduling sorts requests by:
- (a) Start time, increasing
 - (b) Finish time, increasing
 - (c) Length (duration), increasing
 - (d) Number of conflicts, increasing
- B3. (Short answer)** Consider three intervals: $[0, 10]$, $[1, 2]$, and $[3, 4]$. Shortest-interval-first would pick $[1, 2]$ first (and then $[3, 4]$, both compatible with $[1, 2]$ but not $[0, 10]$). What is the size of this shortest-first solution, versus the true optimum?
- B4. (True/False)** The depth of a set of intervals is always a valid lower bound on the number of resources (e.g. classrooms) needed for interval partitioning, regardless of which algorithm is used to construct the assignment.
- B5. (Multiple choice)** In Scheduling to Minimize Lateness, the Earliest-Deadline- First algorithm sorts jobs by:
- (a) Processing time t_i , increasing
 - (b) Deadline d_i , increasing
 - (c) Profit, decreasing
 - (d) Slack $d_i - t_i$, decreasing
- B6. (True/False)** An optimal schedule for Minimize Lateness can always be transformed (without increasing the maximum lateness) into one with no idle time and no inversions.
- B7. (Short answer)** In Belady's Furthest-In-Future caching algorithm, when a cache miss occurs and the cache is full, which page is evicted?
- B8. (Multiple choice)** Least-Recently-Used (LRU) caching is best described as:
- (a) An offline algorithm that requires knowledge of all future requests
 - (b) An online algorithm that uses only the request history seen so far
 - (c) Always exactly as good as Furthest-In-Future
 - (d) Optimal only when $k = 1$
- B9. (Short answer)** For the coin system $\{1, 3, 4\}$, what is the smallest amount for which the greedy algorithm (always take the largest coin $\leq$ remaining amount) fails to produce the minimum number of coins, and how many coins does greedy use vs. the optimum?
- B10. (True/False)** In Job Sequencing with Deadlines and Profits (unit-time jobs, single machine), the greedy algorithm that processes jobs in decreasing order of profit and schedules each job at the latest available free slot at or before its deadline (if any) always achieves the maximum total profit.

Part C: Solutions

Part A

- A1.** A **greedy algorithm** builds up a solution incrementally by repeatedly making the choice that looks best *according to some simple, local criterion* at the current step (e.g. “pick the request that finishes earliest among those still available”). Once a choice is made, the algorithm **never reconsiders or undoes it** – there is no backtracking. This makes greedy algorithms typically very efficient (often $O(n \log n)$ after sorting), but the local criterion does not automatically guarantee a globally optimal result; that requires a separate correctness proof.
- A2.** If the greedy rule is wrong, the algorithm may silently produce a *suboptimal* (or even badly suboptimal) answer on inputs that were not tested – with no error or warning, since the algorithm runs to completion and outputs *some* feasible answer. Testing on a finite set of examples can never rule out the existence of a bad instance that the test set did not happen to include (this week’s exercises construct exactly such counter-examples for several plausible-looking greedy rules, e.g. shortest-interval-first for Interval Scheduling and shortest-job-first for Minimize Lateness). Only a proof – typically via exchange argument or greedy-stays-ahead – rules out *all* possible inputs simultaneously.
- A3.** In **greedy stays ahead**, we compare the greedy solution and an arbitrary optimal solution *step by step* (e.g. the i -th request selected by greedy vs. the i -th request selected by the optimal solution, both sorted in the same order). We prove by induction on i that, in some measurable sense, greedy’s i -th choice is “at least as good” as the optimal solution’s i -th choice (e.g. greedy’s i -th interval finishes no later). This implies greedy can always be extended at least as far as the optimal solution, so greedy’s final solution is at least as large/good as optimal – hence optimal.
- A4.** In the **exchange argument**, we start from an arbitrary optimal solution O and show that if O differs from the greedy solution G in some specific, local way (e.g. O contains an “inversion” that G would never create), we can perform a small local *exchange* (e.g. swap the order of two adjacent jobs, or swap which item is evicted) that transforms O into another feasible solution O' with objective value **no worse** than O ’s. Repeating this process eventually transforms O into a solution that looks like (or is exactly as good as) G , without ever making the objective worse – so G must also be optimal.
- A5. Input:** a set of n requests, each request i specified by a start time s_i and finish time f_i (with $s_i < f_i$), all referring to a single shared resource. A set of requests is **compatible** (feasible) if no two of them overlap in time – i.e. for any two requests i, j in the set, either $f_i \leq s_j$ or $f_j \leq s_i$. The **objective** is to select a compatible set of requests of **maximum size** (maximum number of requests accepted).
- A6.** Three plausible but incorrect heuristics:
- **Earliest start time first.** Counter-example: one very long interval starting at time 0 (e.g. $[0, 100]$) blocks out many short intervals that start slightly later but would together allow far more requests to be scheduled.
 - **Shortest interval (duration) first.** Counter-example: a long interval $[0, 10]$ and two short intervals $[1, 2]$ and $[8, 9]$ (mutually compatible, both compatible with neither overlapping each other but each overlapping $[0, 10]$ differently) – e.g. $[0, 10]$, $[1, 2]$, $[3, 4]$:

shortest-first picks $[1, 2]$ then $[3, 4]$ (size 2, happens to tie optimum here), but with $[0, 10]$, $[4, 5]$, and three short overlapping intervals carefully placed, shortest-first can be made to pick fewer than the optimum (this is explored concretely in the exercises).

- **Fewest conflicts first.** Counter-example: an interval with few apparent conflicts might still “use up” a time slot that many other mutually-compatible short intervals need, leading to a smaller final selection than choosing based on finish time.

A7. Earliest-Finish-Time-First (EFT): sort all requests by finish time f_i in increasing order. Initialize the selected set $A = \emptyset$ and a variable `last_finish` $= -\infty$. For each request i in sorted order, if $s_i \geq \text{last_finish}$ (i.e. it starts after the last selected request finishes), add i to A and set `last_finish` $= f_i$. Sorting by finish time is the key idea because it greedily **preserves the maximum possible remaining time** for future choices: among all requests that could be selected first, the one that finishes earliest leaves the largest “free” interval $[f_i, \infty)$ available for subsequent selections, so it can never be strictly worse than any other first choice.

A8. Setup. Let $G = \{g_1, g_2, \dots, g_k\}$ be the requests selected by EFT, ordered by finish time (so $f(g_1) < f(g_2) < \dots$), and let $O = \{o_1, o_2, \dots, o_m\}$ be an arbitrary optimal compatible set, also ordered by finish time. We claim by induction that for every $i \leq \min(k, m)$, $f(g_i) \leq f(o_i)$.

Base case ($i = 1$): g_1 is the globally earliest-finishing request among *all* requests (EFT picks it first, unconditionally), so $f(g_1) \leq f(o_1)$.

Inductive step: suppose $f(g_i) \leq f(o_i)$. Since o_{i+1} is compatible with o_i (both are in O), $s(o_{i+1}) \geq f(o_i) \geq f(g_i)$. So o_{i+1} is a candidate that EFT could select after g_i (it starts no earlier than g_i finishes). EFT selects g_{i+1} as the earliest-finishing request compatible with g_i among all remaining requests, and o_{i+1} is one such candidate, so $f(g_{i+1}) \leq f(o_{i+1})$.

Conclusion. Suppose for contradiction $k < m$. Then by the claim, $f(g_k) \leq f(o_k)$, and o_{k+1} exists with $s(o_{k+1}) \geq f(o_k) \geq f(g_k)$ – so o_{k+1} is compatible with g_k and EFT had not yet terminated (a compatible request remained), a contradiction (EFT only stops when no compatible request remains). Hence $k \geq m$, and since O is optimal (m is the maximum possible), $k = m$: EFT is optimal.

A9. Input: a set of n requests (intervals), each with a start time s_i and finish time f_i , to be scheduled using some number of identical resources (e.g. classrooms, machines). A **resource assignment** is a function mapping each request to one of the resources. The assignment is **valid** if no two requests assigned to the same resource overlap in time. The **objective** is to find a valid assignment using the **minimum number of resources**.

A10. The **depth** of a set of intervals is the maximum, over all time points t , of the number of intervals that contain t (i.e. the maximum number of intervals that are simultaneously “active”). *Proof that depth is a lower bound:* let d be the depth, achieved at some time t^* by a set of d intervals all containing t^* . Any two of these d intervals overlap (both contain t^*), so they *must* be assigned to *different* resources in any valid assignment. Hence any valid assignment needs at least d distinct resources.

A11. Earliest-Start-Time-First (for partitioning): sort all requests by start time s_i in increasing order. Process requests in this order; for each request, assign it to any resource that is currently free (i.e. not occupied by an overlapping previously-assigned request) – if no resource yet used is free, open a new resource (label) for this request.

Proof it never needs more than depth resources: suppose, for contradiction, that processing request i in sorted order requires opening resource number $d + 1$ (where $d = \text{depth}$), i.e. all d resources opened so far are currently occupied by intervals overlapping i . Each of those d occupying intervals j has $s_j \leq s_i$ (processed earlier, by sorted order) and $f_j > s_i$ (overlaps i , since it is still “occupying” its resource at time s_i). So all d of those intervals, plus i itself, contain the time point s_i – giving $d + 1$ intervals all containing s_i , i.e. $\text{depth} \geq d + 1$, contradicting $\text{depth} = d$. So the algorithm never opens more than $d = \text{depth}$ resources.

Conclusion: the algorithm uses at most depth resources (just proved), and by Q10 no valid assignment can use fewer than depth resources. Hence the algorithm’s assignment, using exactly depth resources (or possibly fewer if some intervals never co-occur, but never more), is optimal.

A12. Input: a set of n jobs to be processed on a single machine (one at a time, in some order), where job i has a processing time t_i and a deadline d_i . A **schedule** is an ordering (permutation) of the jobs, run back-to-back starting at time 0; the **completion time** f_i of job i is the sum of processing times of all jobs up to and including i in the order. The **lateness** of job i is $\ell_i = \max(0, f_i - d_i)$. The **objective** is to find a schedule minimizing the **maximum lateness** $L = \max_i \ell_i$.

A13. Earliest-Deadline-First (EDF): sort jobs by deadline d_i in increasing order, and schedule them back-to-back in that order (ties broken arbitrarily).

Counter-example for Shortest-Job-First (SJF): consider two jobs, job 1 with $(t_1, d_1) = (1, 100)$ and job 2 with $(t_2, d_2) = (10, 10)$. SJF schedules job 1 first (shorter processing time), then job 2: job 1 completes at $f_1 = 1$ (lateness 0, since $d_1 = 100$), job 2 completes at $f_2 = 11$ (lateness $11 - 10 = 1$), so $L_{\text{SJF}} = 1$. EDF schedules job 2 first ($d_2 = 10 < d_1 = 100$), then job 1: job 2 completes at $f_2 = 10$ (lateness 0), job 1 completes at $f_1 = 11$ (lateness $11 - 100 < 0 \Rightarrow 0$), so $L_{\text{EDF}} = 0 < L_{\text{SJF}} = 1$. SJF is strictly worse here.

A14. An inversion is a pair of jobs i, j such that i is scheduled before j but $d_i > d_j$ (i.e. a job with a *later* deadline is scheduled *before* a job with an *earlier* deadline). **Idle time** occurs if the machine is left unused at some point before all jobs are completed, even though jobs remain to be processed (i.e. there is a gap in the schedule before time $\sum_i t_i$).

No idle time: removing any idle gap by shifting all subsequent jobs earlier can only *decrease* (or keep equal) every completion time f_i , hence can only decrease or keep equal every lateness $\ell_i = \max(0, f_i - d_i)$, hence can only decrease or keep equal $L = \max_i \ell_i$. So an optimal schedule with no idle time always exists.

No inversions (exchange argument): suppose an optimal schedule S has an inversion – some pair of *adjacent* jobs i (scheduled first) and j (scheduled right after i) with $d_i > d_j$. Swap i and j to get S' . All other jobs’ completion times are unchanged. Job j now finishes earlier (where i used to finish), so ℓ_j can only decrease. Job i now finishes at the time j used to finish (later than before), but since $d_i > d_j$ and j ’s new completion time equals i ’s *old* completion time which was $\leq \dots$ – concretely: let f be the time both jobs together finish (same in S and S'). In S , i finishes at $f - t_j$ and j at f ; in S' , j finishes at $f - t_i$ and i at f . The lateness of the later-finishing job in each case is $\max(0, f - d_j)$ for S (job j finishes at f , $d_i > d_j$) vs. $\max(0, f - d_i)$ for S' (job i finishes at f). Since $d_i > d_j$, $f - d_i < f - d_j$, so S' ’s “later-finishing-job” lateness is $\leq S$ ’s. The earlier-finishing job’s lateness in each case is no larger than this. Hence $\max(\ell_i, \ell_j)$ does not increase under the swap, so the overall maximum lateness L does not increase. Repeatedly removing inversions this way (each swap strictly

reduces the number of inversions, since it fixes an adjacent inverted pair without creating new inversions elsewhere) terminates with an inversion-free, idle-time-free schedule that is at least as good as S – hence also optimal.

A15. EDF, by construction, sorts jobs by increasing deadline and runs them back-to-back, so it has **no idle time** (jobs run consecutively from time 0) and **no inversions** (jobs are literally ordered by non-decreasing d_i , so no job with a larger deadline precedes one with a smaller deadline). By Q14, *some* optimal schedule has no idle time and no inversions. Any two schedules with no idle time and no inversions must order jobs by non-decreasing deadline (ties may be broken arbitrarily among jobs with equal deadlines, but this does not change any completion times since they all have the same total processing time up to that point, hence the same lateness values) – so they are essentially the same schedule (up to tie-breaking) and achieve the same maximum lateness. Since EDF is such a schedule and an optimal inversion-free, idle-free schedule exists with the same maximum lateness, EDF achieves the optimal maximum lateness L .

A16. Input: a sequence of page requests $r_1, r_2, \dots, r_n$ (each r_t is a page identifier) and a cache of fixed size k (can hold k distinct pages at a time). A **cache miss** occurs at time t if r_t is not currently in the cache (in which case r_t must be loaded into the cache, evicting some other page if the cache is full). The **objective** is to choose, for each miss, which page to evict, so as to **minimize the total number of misses** over the whole sequence.

A17. Furthest-In-Future (FIF / Belady’s algorithm): on a cache miss with a full cache, evict the page in the cache whose *next* occurrence in the remaining request sequence is *furthest in the future* (or evict any page that does not occur again at all, if such a page is in the cache).

Why this is “safe” (high-level exchange argument): take any optimal schedule O . If at this step O evicts a different page than FIF would, consider the page f that FIF would evict (whose next use is furthest away) and the page g that O evicts instead (whose next use is sooner). We can modify O into O' that evicts f instead of g at this step, and “patches up” the schedule afterward (essentially, O' keeps g in cache a bit longer and evicts it later, exactly when O would have needed to bring it back). Because f ’s next use is *later* than g ’s, this patch can never cause *more* misses than O had – O' is feasible and has at most as many misses as O . Repeating this argument at every step where FIF and O disagree converts O into a schedule that behaves like FIF, without ever increasing the miss count – so FIF is optimal.

A18. Least-Recently-Used (LRU): on a cache miss with a full cache, evict the page in the cache that was *least recently requested* (i.e. whose most recent access is furthest in the *past*). LRU is a reasonable *online* heuristic because it requires *no knowledge of future requests* – only the history of requests seen so far – which matches real systems where future access patterns are unknown. It is based on the (often reasonable) assumption of *temporal locality*: pages used recently are likely to be used again soon, so pages not used recently are good eviction candidates.

Example where LRU and FIF differ: cache size $k = 2$, sequence $0, 1, 2, 0, 1, 2, 0, 1, 2, \dots$ (cyclic). FIF, knowing the future, can always evict the page whose next use is furthest away (e.g. at each miss, keep the two pages that will be needed soonest), achieving far fewer misses than LRU, which – having no future information – ends up evicting the page that is needed *immediately next* in this adversarial cyclic pattern, causing a miss on (almost) every request. (This is exactly the instance constructed in the Problem 18 exercise.)

- A19.** A coin system is **canonical** if, for *every* target amount $V \geq 0$, the greedy algorithm (repeatedly subtract the largest denomination $\leq$ the remaining amount) produces a representation using the *minimum possible* number of coins. The system $\{1, 5, 10, 25\}$ is canonical: one can check (or prove via case analysis on the remainder modulo each denomination) that greedy’s choice at each step never forces extra coins later, for any amount. The system $\{1, 3, 4\}$ is **not** canonical: the smallest counter-example is $V = 6$. Greedy takes 4, then 1, then 1 – three coins ($4 + 1 + 1 = 6$). But the optimum is two coins: $3 + 3 = 6$.
- A20. Greedy algorithm:** sort jobs in *decreasing* order of profit p_i . Maintain an array of time slots $1, 2, \dots, D$ (where $D = \max_i d_i$), all initially free. For each job i in this order, find the *latest* free slot $t \leq d_i$ (if any); if one exists, assign job i to slot t (marking it occupied) and add p_i to the total profit; otherwise, skip job i (it cannot be scheduled profitably).
- Why decreasing profit order is right:* we want to “lock in” the highest-profit jobs first while they still have a free slot available at or before their deadline. *Why “as late as possible”:* placing a job in the latest available slot at or before its deadline preserves all *earlier* slots for jobs with *tighter (earlier) deadlines* that may be considered later – it never “wastes” an early slot on a job that had more flexibility. Together, these ensure that whenever a later (lower-profit) job cannot be placed, it is genuinely because all slots up to its deadline are occupied by jobs of equal or greater profit – which is unavoidable in any schedule.
- A21.** All four proofs share the same high-level structure: (1) define a precise *measure of progress* or *structural property* that the greedy solution satisfies at every step (earliest finish time so far / using $\leq$ depth resources / no inversions and no idle time / evicting the furthest-future page); (2) show that *any* solution – in particular, an optimal one – can be transformed, *without making it worse*, to also satisfy this property, or show directly that greedy’s measure is always at least as good as optimal’s at every step (greedy stays ahead); (3) conclude that greedy’s final objective value matches (or is no worse than) the optimal’s. In every case the proof is *local* (about one step or one exchange) but the conclusion is *global* (about the entire solution), connected via induction.
- A22.** A systematic process: (1) **Try to break it first.** Before trying to prove the rule correct, actively search for counter-examples: construct small instances (2–4 items) designed to create *ties* or *near-ties* under the greedy criterion, since these are where greedy’s “local” view is most likely to miss a better global choice. (2) **Look for structural invariants.** If small/random instances all agree with brute force, try to identify *what quantity* greedy is implicitly maximizing/minimizing at each step, and check whether that quantity provably dominates the corresponding quantity for any other solution (greedy stays ahead). (3) **Try an exchange.** Take a hypothetical optimal solution that disagrees with greedy at the first point of difference, and check whether swapping/adjusting it to agree with greedy at that point can be shown to never hurt the objective. (4) **Stress test computationally.** Generate many random small instances, compute both the greedy answer and a brute-force optimum, and compare – exactly as Problems 5, 9, 13, and 19 do this week. If a discrepancy is found, minimize the counter-example by hand to understand exactly which feature of the instance breaks the rule (often a tie, an extreme length, or a cyclic pattern).

Part B

- B1. False.** A defining property of greedy algorithms is that they make each choice *irrevocably* – there is no backtracking or revision of earlier decisions.

- B2. (b) Finish time, increasing.** EFT sorts by finish time f_i in increasing order; this is the key to the correctness proof (Q7–Q8).
- B3.** Shortest-interval-first picks $[1, 2]$, then $[3, 4]$ (both compatible with each other; $[0, 10]$ is excluded since it overlaps both), giving a solution of size 2. The true optimum is also 2 (e.g. $\{[1, 2], [3, 4]\}$, since $[0, 10]$ alone only gives size 1) – so on this particular instance shortest-first happens to tie the optimum. (A larger/different instance is needed to make shortest-first *strictly* worse, as explored in Problem 6.)
- B4. True.** The depth lower bound (Q10) is a property of the *instance* itself (the maximum number of pairwise-overlapping intervals at any point in time) and does not depend on which algorithm is used – it applies to *any* valid assignment.
- B5. (b) Deadline d_i , increasing.** EDF sorts by deadline; processing time and profit are irrelevant to this algorithm (profit doesn’t even appear in the Minimize Lateness problem).
- B6. True.** This is exactly the content of Q14 – idle time can be removed without increasing lateness, and inversions can be removed one at a time via an adjacent swap that does not increase the maximum lateness.
- B7.** The page in the cache whose **next request (use) is furthest in the future** among all pages currently in the cache – or any page that is never requested again, if one is present.
- B8. (b) An online algorithm that uses only the request history seen so far.** LRU makes eviction decisions based on *past* access times only, with no knowledge of future requests – unlike FIFO, which is an offline algorithm requiring the full future sequence.
- B9.** $V = 6$. Greedy uses **3** coins ($4 + 1 + 1$); the optimum uses **2** coins ($3 + 3$).
- B10. True.** This greedy strategy (decreasing profit, place each job as late as possible before its deadline) is a classic exchange-argument result and always achieves the maximum total profit for unit-time Job Sequencing with Deadlines and Profits.